

Linguagem Técnica de Programação Orientada a Objetos

AULA 2 – Revisão de conceitos básicos - Tipos Primitivos

Prof. Wanderson Santos

wanderson.santos@projecao.br



Roteiro da Aula



- Revisão de conceitos básicos
 - Tipos de Dados Primitivos
 - Literais
 - Literais em Ponto Flutuante
 - Literais caracteres ou Strings
 - Declaração de variáveis e constantes
 - Operadores da Linguagem Java
 - Operador de Atribuição
 - Operadores unário
 - Operadores de Incremento e Decremento
 - Operadores Unário Mais e Menos
 - Operador de Negação
 - Operadores Aritméticos, de comparação, lógicos, condicional (ternário), precedência e escopo.
-



Tipos de Dados Primitivos



- Java é **fortemente tipada** e possui **tipos estáticos**, ou seja, antes de usar qualquer variável será obrigatório declarar a variável e seu tipo.
 - A Linguagem Java, assim como outras linguagens, possui um conjunto de tipos necessários para as construções básicas da linguagem.
-



Tipos de Dados Primitivos



- Este conjunto é chamado de **Tipos Primitivos** e na prática são implementados por palavras-chave.
 - Cada tipo primitivo possui um tamanho de memória (**em bits**) que é usado para armazenar o seu valor.
-

Tipos de Dados Primitivos

- Segue escala de valores dos Tipos do Java:

TIPO	TAMANHO (BITS)	ESCALA
boolean	-	true ou false
char	16	'\u0000' a '\uFFFF' (0 a 65535 – caracteres Unicode ISO)
byte	8	-128 a +127
short	16	-32768 a 32767
int	32	-2.147.483.648 e 2.147.483.647

Tipos de Dados Primitivos

- Segue escala de valores dos Tipos do Java:

TIPO	TAMANHO (BITS)	ESCALA
long	64	9223372036854775808 a 9223372036854775807
float	32	1,40129846432481707e-45 a 3,40282346638528860e+38
double	64	4,94065645841246544e-324d a 1,79769313486231570e+308d

Tipos de Dados Primitivos

- Um pouco mais sobre os tipos:
 - **boolean**: o boolean é fácil, ele só pode armazenar dois valores: **true ou false**. Java só aceita estes dois valores. Outras linguagens aceitam 1 ou 0, V ou F, **mas Java não!!!!**
 - **char**: este é tranquilo também. O char guarda qualquer caracter Unicode.
 - Unicode representa dados relacionados a texto, incluindo letras, símbolos e caracteres especiais
-

Tipos de Dados Primitivos

- Um pouco mais sobre os tipos:
 - **byte**: o byte aceita números compreendidos entre -127 e 128. É possível usar um modificador para usarmos somente números positivos também.
 - **short**: o propósito do short é o mesmo do byte, porém ele guarda o dobro de valores do byte.
 - **int**: este tipo guarda somente valores inteiros.
-



Tipos de Dados Primitivos



- **float e double:** estes dois tipos fazem a mesma coisa: guardam números decimais (também chamados de **Ponto Flutuante**). A diferença é que o **double**, naturalmente, guarda valores maiores que o **float**. Eles são usados em programas que requerem grande precisão de contas e resultados.
-

Tipos de Dados Primitivos

- Quando declaramos uma variável e não atribuímos nenhum valor a ela, o **Java** atribui um valor automaticamente a ela de acordo com alguns padrões conforme é mostrado abaixo e no próximo slide:

TIPO	VALOR PADRÃO
BYTE	0
SHORT	0
INT	0

Tipos de Dados Primitivos

- Valores padrões para os Tipos em Java:

LONG	0L
FLOAT	0.0f
DOUBLE	0.0d
CHAR	'\u0000'
BOOLEAN	false

Tipos de Dados Primitivos

- **Literais**

- Um literal é uma representação de algum valor fixo que o programador **literalmente** estabelece. Cada tipo primitivo possui um literal correspondente e nos slides anteriores foram representados os literais padrões para cada tipo. Observem os exemplos a seguir. São todos exemplos de literais:

```
boolean resultado = true;  
char cMaiusculo = 'C';  
byte b = 100;  
short s = 10000;  
int i = 100000;
```

Tipos de Dados Primitivos

- E se pegássemos como exemplo o **literal 100**? Ele é **int** ou **long**? Você já viu que os valores **long**, **float** e **double** tem letras no final do valor e estas letras servem para distinguir o tipo do literal em algumas situações.
 - Portanto, se 100 é um long, podemos representar como 100L ou 100l (é preferível usar o “L” maiúsculo). Se o 100 não tiver uma letra no final, será considerado um **int**.
-

Tipos de Dados Primitivos

- Os valores literais para **byte**, **short**, **int** e **long** podem ser criados a partir de um literal **int**.
 - Valores do tipo **long** que excedem a capacidade de um **int** pode ser criados por meio de literais do tipo **long**.
 - Os literais do tipo **int** podem ser **decimais**, **binários** ou **hexadecimais**. Esses dois últimos sistemas de numeração são usados em programas que lidam e manipulam endereços de memória por exemplo. Vejam os exemplos a seguir no próximo slide.
-

Tipos de Dados Primitivos

- Representações dos literais:

```
// O número 26 em decimal  
int decimal = 26;  
  
// Em hexadecimal  
int hexa = 0x1a;  
  
// E em binário  
int binario = 0b11010;
```

Tipos de Dados Primitivos

- **Literais em Ponto Flutuante**
 - Um literal em Ponto Flutuante é do tipo **float** se terminar com a letra “F” ou “f”, caso contrário, seu tipo será **double** e pode terminar com “D” ou “d” opcionalmente.
 - Os tipos de Ponto Flutuante (float ou double) também podem ser expressos na notação científica usando a letra “E” ou “e”, “F” ou “f” para um literal float e “D” ou “d” para um literal double. Observem os exemplos no próximo slide.
-



Tipos de Dados Primitivos



- Exemplo dos Literais em Ponto Flutuante:

```
double d1 = 123.4;  
// é o mesmo valor que d1 mas em notação científica  
double d2 = 1.234e2;  
float f1  = 123.4f;
```

Tipos de Dados Primitivos

- **Literais caracteres ou Strings**
 - O literais dos tipos **char** e **String** podem conter qualquer caracter **Unicode**. Dependendo do editor de texto que você estiver usando para programar você pode digitar diretamente caracteres como “Ç”, “ã” e outros ou usar uma sequência chamada “**escape para Unicode**” para poder gerá-los. Vejam os exemplos no próximo slide.
-

Tipos de Dados Primitivos

- Exemplos de literais de caracteres:

```
//literal char
```

```
char c = 'a'; //observe que o literal char tem que estar entre aspas simples!
```

```
char n = '\u004E'; //corresponde à letra E
```

Tipos de Dados Primitivos

- As strings em Java na verdade são instâncias de uma classe Java chamada **String**. Existem várias maneiras de se representar uma string em Java e todas elas usam aspas duplas para a string desejada.
- Existe também o **literal nulo**, representado pela palavra-chave **null**. O literal nulo representa ausência de valor e dado.

```
//exemplo de literal String  
//observe que a classe String começa com S maiúsculo!  
  
String s = "Exemplo de uma string";
```

Declaração de variáveis

- Antes de **usar uma variável** em Java, deve-se criá-la declarando o seu nome e o seu tipo.

```
public static void main (String arguments[]) {  
    int maiorValor, menorValor;  
    String nomeUsuario;  
    boolean fimJogo = false;  
    ...  
}
```

- **Podem ser declaradas em qualquer lugar dentro do método !!**
-



Declaração de variáveis



- Variáveis em Java são sempre definidas **dentro de um escopo**.
 - Não existem variáveis globais.
 - Reunir variáveis em um mesmo lugar, dando a elas nomes significativos, facilita ao leitor entender o que o programa faz.
 - Uma seção de declarações de variáveis encoraja o planejamento do programa antes de começar a escrevê-lo.
-

Declaração de variáveis

- Java é uma linguagem **case-sensitive**, ou seja, ela diferencia letras maiúsculas e minúsculas.
 - Ex: nome, Nome, nOMe são distintos.
 - O uso de identificadores significativos é uma boa prática de programação.
 - 51 Palavras Reservadas em Java Não podem ser usadas como identificadores. Seguem alguns exemplos:
 - abstract, boolean, break, byte, case, catch, char, class,... , else, extends, false, final...for...if....
-

Declaração de variáveis

- Em Java, toda variável tem um tipo que não pode ser mudado, uma vez que declarado:
 - **tipoDaVariavel nomeDaVariavel;**
 - Por exemplo, é possível ter uma idade que vale um número inteiro:
 - **int idade;**
-

Declaração de variáveis

- A linha a seguir é a tradução de “idade deve valer agora quinze”.
 - **idade = 15;**
 - Além de atribuir, você pode utilizar esse valor.
 - `//declara a idade`
 - `int idade;`
 - `// imprime a idade`
 - **`System.out.println(idade);`**
-

Declaração de variáveis

- Podemos utilizar o valor de uma variável alterar ou definir uma segunda variável.
 - `int idadeNoAnoQueVem;`
 - **`idadeNoAnoQueVem = idade + 1;`**
 - No momento que você declara uma variável, também é possível inicializá-la por praticidade:
 - **`int idade = 15;`**
-

Declaração de Constantes

- As constantes em Java são declaradas através do modificador **final**.
 - A sintaxe para declaração de uma constante é:
 - **final int limite = 10;**
 - O exemplo acima cria uma constante chamada “limite” com tipo de dado **int** e com valor 10.
 - Esse valor não pode ser alterado.
 - Cada vez que o identificador limite for acessado no programa, seu valor será entendido como 10 inteiro.
-

Operadores da Linguagem Java

- Os principais operadores da linguagem Java são os:

- Unários
 - Aritméticos
 - de Comparação
 - Lógicos (ou Booleanos)
 - de Atribuição
 - Condicionais
-

Operadores da Linguagem Java

- **Operador de Atribuição**
 - Após uma variável ser declarada, ela pode ser inicializada com diferentes valores. Para isso, usamos o operador de atribuição =

Exemplos:

- `int x = 10;`
 - `boolean condicao = false;`
 - `char c = 'A';`
 - `long l = 1234L;`
 - `float f = 1.23f;`
 - `double d = 1.23d;`
-

Operadores da Linguagem Java

- **Operadores unário**
 - Os principais operadores unário do Java são aplicados especificamente sobre um operador.
 - Eles realizam alguns trabalhos básicos como incremental, decremental e inversão de valores numéricos e booleanos::

Operadores de incremento e decremento: ++ , --

Operadores mais e menos: + , -

Operador de negação: !

Operadores da Linguagem Java

- Operadores de Incremento e Decremento

Os operadores para incremento e decremento são, respectivamente, `++` e `--`.

Tais operadores podem ser usados na forma pós-fixada (`x++`) ou pré-fixada (`++x`).

A diferença só existe quando o operador aparece em expressões. A forma pré-fixada faz a adição/subtração primeiro; a pós-fixada avalia o valor antigo, depois incrementa/decrementa a variável.

Operadores da Linguagem Java

- Exemplos e Operadores de Incremento e Decremento:

Valor inicial de x	Expressão	Valor final de y	Valor final de x
5	<code>y = x++;</code>	5	6
5	<code>y = ++x;</code>	6	6
5	<code>y = x--;</code>	5	4
5	<code>y = --x;</code>	4	4

Operadores da Linguagem Java

- **Operadores Unário Mais e Menos**

Os operadores unários `+` e `-` somente indicam se o valor é positivo ou negativo, respectivamente.

Tais operadores não se aplicam a booleanos.

Podem ser aplicados a expressões e a variáveis do tipo `char`.

Exemplos:

- `char c = 10;`
 - `int i = -c;`
 - `int j = - (i + 6);`
-

Operadores da Linguagem Java

- Operadores Unário Mais e Menos

Os operadores unários `+` e `-` promovem a expressão para `int`.

Sendo assim, o trecho de código abaixo apresentaria um erro na linha 3:

```
1. byte b1 = 1;  
2. byte b2 = 127;  
3. byte b3 = -b1; //erro: o operador - promoveu a  
                  //expressão para int
```

Operadores da Linguagem Java

- Operador de Negação

O operador unário **!** causa a negação de uma expressão booleana.

Sendo assim, só se aplica a variáveis e expressões booleanas.

Exemplo:

```
boolean a, b, c;
```

```
a = true;
```

```
b = !a;
```

```
c = !(a && b);
```

Resultado:

```
c = true
```

Operadores da Linguagem Java

- **Operadores Aritméticos**

Os operadores aritméticos habituais: `+`, `-`, `*` e `/`, são usados para adição, subtração, multiplicação e divisão, respectivamente.

O operador `/` denota divisão inteira, se os dois argumentos forem inteiros, e divisão de ponto flutuante, caso contrário.

Existe ainda o operador `%` usado para se obter o resto da divisão inteira entre dois números.

Operadores da Linguagem Java

- **Operadores Aritméticos**

Os operadores aritméticos aplicam-se a todos os tipos primitivos numéricos e ao tipo char.

A divisão inteira por zero (incluindo a operação de módulo) gera uma `ArithmeticException`.

Divisões por zero incluindo tipos ponto-flutuante não geram exceções.

O uso dos operadores aritméticos promovem a expressão para, no mínimo, `int`.

Operadores da Linguagem Java

- Operadores de Comparação

Os operadores de igualdade e desigualdade em Java são, respectivamente, `==` e `!=`.

Exemplos:

- `(3 == 4)` resulta em `false`
- `(3 != 4)` resulta em `true`

Observe que esses operadores também podem ser aplicados a operandos booleanos.

Operadores da Linguagem Java

- Operadores de Comparação

Os outros operadores de comparação são:

- `>` maior que
- `>=` maior ou igual que
- `<` menor que
- `<=` menor ou igual que

Observe que não faz sentido aplicar tais operadores a operandos booleanos.

O resultado de uma comparação em Java é sempre um valor booleano.

Operadores da Linguagem Java

- Operadores Lógicos

Os operadores lógicos em Java são:

- `&&` e lógico
- `||` ou lógico

Esses operadores são aplicáveis somente a operandos ou expressões booleanas e o resultado também é sempre booleano.

Operadores da Linguagem Java

- Operadores Lógicos

Com o uso dos operadores `&&` e `||` é possível obter o resultado sem que a expressão completa tenha sido avaliada, isto é, fazendo somente uma avaliação parcial.

Exemplo:

`(a && b)`

Se `a` for avaliado como falso, então `b` não precisa e não será avaliado, pois se `a` é falso já é possível afirmar que `a && b` é falso.

Operadores da Linguagem Java

- **Operador Condicional (Ternário)**

Existe um operador condicional em Java que sintetiza a função do comando `if/else`.

Esse operador é `?`, que é considerado um operador ternário por necessitar de três operandos.

Exemplo:

- `a ? b : c;`
 - o primeiro operando, no caso `a`, deve ser um valor ou expressão booleana
 - se `a` for verdadeiro, o resultado será `b`
 - caso contrário, o resultado será `c`
-

Operadores da Linguagem Java

- Exemplo de uma Condicional Ternário:

Exemplo:

```
if (Character.isUpperCase(aChar)) {  
    System.out.println("O caractere " + aChar + " é upper case.");  
} else {  
    System.out.println(" O caractere " + aChar + " é lower case.");  
}
```

Usando o operador condicional o código ficaria assim:

```
System.out.println(" O caractere " + aChar + " é " +  
    (Character.isUpperCase(aChar) ? "upper" : "lower") + "case.");
```

Operadores da Linguagem Java

- **Precedência de Operadores Java**

Ordem descendente de precedência:

Categoria	Operador
Unários	++ -- + - !
Aritméticos	* / % + -
Comparação	< <= > >= == !=
Lógicos	&&
Condicional	? :
Atribuição	=

Operadores da Linguagem Java

- **Escopo de um bloco:**

O controle de fluxo de aplicações Java é organizado em blocos. Um bloco ou conjunto de instruções/comandos em Java é delimitado por { e }.

Uma variável declarada dentro de um bloco é visível somente dentro do bloco.

Exemplo:

```
...  
{  
    int i = 0;  
    i = i + x;  
    ...  
}  
x = i;    // erro, pois i foi declarada dentro do bloco anterior a  
          // essa linha, e não é visível fora do bloco.
```

Comandos de entrada e saída

- Vamos conhecer a Classe **Scanner**
 - Em linguagens de programação estruturada, como C e Pascal, a entrada de dados usando o teclado é realizada utilizando subprogramas desenvolvidos para esta finalidade. Em C, por exemplo, tem-se a função **scanf()** e em Pascal, o procedimento **readln()**.
 - Na linguagem Java, a partir do Java 1.5 ou Java 5.0, o pacote de classes **java.util** disponibilizou a classe **Scanner**, que implementa operações de entrada de dados pelo teclado.
-

Comandos de entrada e saída

- A classe Scanner possui vários **métodos** que possibilitam a **entrada de dados de diferentes tipos**, entre eles destacam-se:
 - String **next()** - retorna uma cadeia de caracteres simples, ou seja, que não usa o caractere espaço em branco;
 - double **nextDouble()** - retorna um número em notação de ponto flutuante normalizada em precisão dupla de 64 bits (usado para receber valores reais ou monetários);
 - int **nextInt()** - retorna um número inteiro de 32 bits;
-



Comandos de entrada e saída



- boolean **hasNextDouble()** - retorna true se o próximo dado de entrada pode ser interpretado como um valor double;
 - boolean **hasNextInt()** - retorna true se o próximo dado de entrada pode ser interpretado como um valor int;
 - String **nextLine()** - retorna uma cadeia de caracteres, por exemplo: “LTP Orientada a Objetos - Java”;
 - long **nextLong()** - retorna um número inteiro de 64 bits.
-

Comandos de entrada e saída

- Para utilizar a classe **Scanner** em uma aplicação **Java** deve-se proceder da seguinte maneira:

[1]	importar o pacote java.util: <u>import</u> java.util.Scanner;
[2]	Instanciar e criar um objeto Scanner: Scanner ler = <u>new</u> Scanner(System.in);

Comandos de entrada e saída

[3]	Lendo valores através do teclado:
[3.1]	Lendo um valor inteiro: <pre><u>int</u> n;</pre> System.out.printf("Informe um número para a tabuada: "); n = ler.nextInt();

Comandos de entrada e saída

```
Informe o número da tabulada: 5  
O valor é: 5
```

Comandos de entrada e saída

```
Exemplo01.java ×
1 /**
2  * Aula 02 - Exemplos
3  * @author Wanderson Santos
4  */
5 package aula02;
6
7 import java.util.Scanner; // Importar o pacote Java
8
9 public class Exemplo01 {
10     public static void main(String[] args) {
11         // Instanciar o objeto Scanner
12         Scanner ler = new Scanner(System.in);
13
14         // Criando a variável
15         int n;
16
17         // Recebendo dados do usuário
18         System.out.printf("Informe o número da tabuada: ");
19         n = ler.nextInt();
20
21         System.out.println("O valor é: " + n);
22
23         ler.close(); // fechar o recurso do objeto Scanner
24     }
25 }
26
```

Comandos de entrada e saída

[3.2]

Lendo um valor real:

```
float preco;
```

```
System.out.printf("Informe o preço da mercadoria = R$ ");
```

```
preco = ler.nextFloat();
```

Comandos de entrada e saída

```
Informe o preço da mercadoria[R$]: 35,99  
O valor é: 35.99
```

Comandos de entrada e saída

```
Exemplo02.java ×
1  /**
2   * Aula 02 - Exemplos
3   * @author Wanderson Santos
4   */
5  package aula02;
6
7  import java.util.Scanner; // Importar o pacote Java
8
9  public class Exemplo02 {
10     public static void main(String[] args) {
11         // Instanciar o objeto Scanner
12         Scanner ler = new Scanner(System.in);
13
14         // Criando a variável
15         float preco;
16
17         // Recebendo dados do usuário
18         System.out.printf("Informe o preço da mercadoria[R$]: ");
19         preco = ler.nextFloat();
20
21         System.out.println("O valor é: " + preco);
22
23         ler.close(); // fechar o recurso do objeto Scanner
24     }
25 }
26
```

Comandos de entrada e saída

[3.3]

Lendo um valor real:

```
double salario;
```

```
System.out.printf("Informe o salário do Funcionário = R$ ");
```

```
salario = ler.nextDouble();
```

Comandos de entrada e saída

```
Informe o salário do Funcionário [R$]: 5970  
O valor é: 5970.0
```

Comandos de entrada e saída

```
Exemplo03.java ×
1 /**
2  * Aula 02 - Exemplos
3  * @author Wanderson Santos
4  */
5 package aula02;
6
7 import java.util.Scanner; // Importar o pacote Java
8
9 public class Exemplo03 {
10     public static void main(String[] args) {
11         // Instanciar o objeto Scanner
12         Scanner ler = new Scanner(System.in);
13
14         // Criando a variável
15         double salario;
16
17         // Recebendo dados do usuário
18         System.out.printf("Informe o salário do Funcionário [R$]: ");
19         salario = ler.nextFloat();
20
21         System.out.println("O valor é: " + salario);
22
23         ler.close(); // fechar o recurso do objeto Scanner
24     }
25 }
26
```

Comandos de entrada e saída

[3.4]

Lendo uma `String`, usado na leitura de palavras simples que não usam o caractere de espaço (ou barra de espaço):

```
String s;
```

```
System.out.printf("Informe uma palavra simples:\n");
```

```
s = ler.next();
```

Comandos de entrada e saída

[3.4]

Lendo uma String, usado na leitura de palavras simples que não usam o caractere de espaço (ou barra de espaço):

```
String s;
```

```
System.out.printf("Informe uma palavra simples:\n");
```

```
s = ler.next();
```

Comandos de entrada e saída

```
Informe uma palavra simples: casa  
O valor é: casa
```

Comandos de entrada e saída

Exemplo04.java

```
1 /**
2  * Aula 02 - Exemplos
3  * @author Wanderson Santos
4  */
5 package aula02;
6
7 import java.util.Scanner; // Importar o pacote Java
8
9 public class Exemplo04 {
10     public static void main(String[] args) {
11         // Instanciar o objeto Scanner
12         Scanner ler = new Scanner(System.in);
13
14         // Criando a variável
15         String s;
16
17         // Recebendo dados do usuário
18         System.out.printf("Informe uma palavra simples: ");
19         s = ler.next();
20
21         System.out.println("O valor é: " + s);
22
23         ler.close(); // fechar o recurso do objeto Scanner
24     }
25 }
26
```

Comandos de entrada e saída

Agora, digite a palavra: casa grande

```
Informe uma palavra simples: casa grande  
O valor é: casa
```

Comandos de entrada e saída

```
Exemplo04.java ×
1 /**
2  * Aula 02 - Exemplos
3  * @author Wanderson Santos
4  */
5 package aula02;
6
7 import java.util.Scanner; // Importar o pacote Java
8
9 public class Exemplo04 {
10     public static void main(String[] args) {
11         // Instanciar o objeto Scanner
12         Scanner ler = new Scanner(System.in);
13
14         // Criando a variável
15         String s;
16
17         // Recebendo dados do usuário
18         System.out.printf("Informe uma palavra simples: ");
19         s = ler.next();
20
21         System.out.println("O valor é: " + s);
22
23         ler.close(); // fechar o recurso do objeto Scanner
24     }
25 }
26
```


Comandos de entrada e saída

[3.5]

Lendo uma String, usado na leitura de palavras compostas, por exemplo, Pato Branco:

```
String s;
```

```
System.out.printf("Informe uma cadeia de caracteres:\n");
```

```
s = ler.nextLine();
```

Comandos de entrada e saída

[3.5]

Lendo uma String, usado na leitura de palavras compostas, por exemplo, Pato Branco:

```
String s;
```

```
System.out.printf("Informe uma cadeia de caracteres:\n");
```

```
s = ler.nextLine();
```



Comandos de entrada e saída



```
Informe uma cadeia de caracteres: casa azul  
O valor é: casa azul
```

Comandos de entrada e saída

```
Exemplo05.java ×
1 /**
2  * Aula 02 - Exemplos
3  * @author Wanderson Santos - professor@wandersonsantos.com
4  */
5 package aula02;
6
7 import java.util.Scanner; // Importar o pacote Java
8
9 public class Exemplo05 {
10     public static void main(String[] args) {
11         // Instanciar o objeto Scanner
12         Scanner ler = new Scanner(System.in);
13
14         // Criando a variável
15         String s;
16
17         // Recebendo dados do usuário
18         System.out.printf("Informe uma cadeia de caracteres: ");
19         s = ler.nextLine();
20
21         System.out.println("O valor é: " + s);
22
23         ler.close(); // fechar o recurso do objeto Scanner
24     }
25 }
26
```

Comandos de entrada e saída

```
Exemplo05.java ×
1 /**
2  * Aula 02 - Exemplos
3  * @author Wanderson Santos - professor@wandersonsantos.com
4  */
5 package aula02;
6
7 import java.util.Scanner; // Importar o pacote Java
8
9 public class Exemplo05 {
10     public static void main(String[] args) {
11         // Instanciar o objeto Scanner
12         Scanner ler = new Scanner(System.in);
13
14         // Criando a variável
15         String s;
16
17         // Recebendo dados do usuário
18         System.out.printf("Informe uma cadeia de caracteres: ");
19         s = ler.nextLine();
20
21         System.out.println("O valor é: " + s);
22
23         ler.close(); // fechar o recurso do objeto Scanner
24     }
25 }
26
```

Comandos de entrada e saída

- Na leitura consecutiva de valores numéricos e **String** deve-se esvaziar o buffer do teclado antes da leitura do valor String, por exemplo:

[3.6]

```
int n;  
  
String s;  
  
System.out.printf("Informe um Número Inteiro: ");  
n = ler.nextInt();  
  
ler.nextLine(); // esvazia o buffer do teclado  
  
System.out.printf("Informe uma cadeia de caracteres:\n");  
s = ler.nextLine();
```

Comandos de entrada e saída

- Identifique o problema!

```
Informe um Número Inteiro: 34  
O valor é: 34  
Informe uma cadeia de caracteres: O valor é:
```

Comandos de entrada e saída

```
Exemplo06.java ×
1  /**
2   * Aula 02 - Exemplos
3   * @author Wanderson Santos - professor@wandersonsantos.com
4   */
5  package aula02;
6
7  import java.util.Scanner; // Importar o pacote Java
8
9  public class Exemplo06 {
10     public static void main(String[] args) {
11         // Instanciar o objeto Scanner
12         Scanner ler = new Scanner(System.in);
13
14         // Criando a variável
15         int n;
16         String s;
17
18         // Recebendo dados do usuário
19         System.out.printf("Informe um Número Inteiro: ");
20         n = ler.nextInt();
21         System.out.println("O valor é: " + n);
22
23         System.out.printf("Informe uma cadeia de caracteres: ");
24         s = ler.nextLine();
25         System.out.println("O valor é: " + s);
26
27         ler.close(); // fechar o recurso do objeto Scanner
28     }
29 }
30
```


Comandos de entrada e saída

```
Exemplo06b.java ×
1 /**
2  * Aula 02 - Exemplos
3  * @author Wanderson Santos - professor@wandersonsantos.com
4  */
5 package aula02;
6
7 import java.util.Scanner; // Importar o pacote Java
8
9 public class Exemplo06b {
10     public static void main(String[] args) {
11         // Instanciar o objeto Scanner
12         Scanner ler = new Scanner(System.in);
13
14         // Criando a variável
15         int n;
16         String s;
17
18         // Recebendo dados do usuário
19         System.out.printf("Informe um Número Inteiro: ");
20         n = ler.nextInt();
21         System.out.println("O valor é: " + n);
22
23         ler.nextLine(); // esvaziar o buffer do teclado
24
25         System.out.printf("Informe uma cadeia de caracteres: ");
26         s = ler.nextLine();
27         System.out.println("O valor é: " + s);
28
29         ler.close(); // fechar o recurso do objeto Scanner
30     }
31 }
```

Comandos de entrada e saída

- Problema Solucionado!

```
Informe um Número Inteiro: 34  
O valor é: 34  
Informe uma cadeia de caracteres: casa amarela  
O valor é: casa amarela
```

Comandos de entrada e saída

[3.7]

Lendo um caractere usando o método `read()` do pacote de classes `System.in`:

```
public static void main(String args[]) throws Exception {
```

```
    char c;
```

```
    System.out.printf("Informe um Caractere: ");
```

```
    c = (char)System.in.read();
```

```
}
```

Comandos de entrada e saída

```
Informe um Character: c  
O valor é: c
```

Comandos de entrada e saída

```
Exemplo07.java ×
1 /**
2  * Aula 02 - Exemplos
3  * @author Wanderson Santos - professor@wandersonsantos.com
4  */
5 package aula02;
6
7 import java.util.Scanner; // Importar o pacote Java
8
9 public class Exemplo07 {
10     public static void main(String[] args) {
11         // Instanciar o objeto Scanner
12         Scanner ler = new Scanner(System.in);
13
14         // Criando a variável
15         char c;
16
17         // Recebendo dados do usuário
18         System.out.printf("Informe um Caractere: ");
19         // Lê a String e pega o caractere na posição zero
20         c = ler.next().charAt(0);
21
22         System.out.println("O valor é: " + c);
23
24         ler.close(); // fechar o recurso do objeto Scanner
25     }
26 }
27
```

Comandos de entrada e saída

```
Informe um Character: c  
O valor é: c
```

```
Informe um Character: casa  
O valor é: c
```

Comandos de entrada e saída

```
Informe um Character: c  
O valor é: c
```

```
Informe um Character: casa  
O valor é: c
```

Comandos de entrada e saída

- Método `printf()`
 - A partir do Java 5.0 teve a incorporação da função **`printf()`** do **C** como um método do pacote de classes **`System.out`**.
 - O método `printf()`, utilizado para realizar uma saída de dados no fluxo de saída padrão `System.out`, tem a seguinte forma geral:

```
System.out.printf(<em>expressão_de_controle</em>, argumento1, argumento2, ...);
```

Comandos de entrada e saída

- A **expressão_de_controle** deve ser uma sequência de caracteres (portanto, delimitada por aspas duplas) que determina as informações que serão mostradas na tela.
 - Nesta expressão podem existir dois tipos de informações:
 - **caracteres comuns e códigos de controle** (ou especificadores de formato). Os códigos de controle, mostrados na Tabela no próximo slide, são precedidos por um % e são aplicados aos argumentos na mesma ordem em que aparecem. É importante destacar que deve existir para cada código de controle um argumento correspondente.
-

Comandos de entrada e saída

Código	Formato (tipo de dados)
%c	Caractere simples (char)
%s	Cadeia de caracteres (String)
%d	Inteiro decimal com sinal (int)
%i	Inteiro decimal com sinal (int)

Comandos de entrada e saída

<code>%ld</code>	Inteiro decimal longo (long)
<code>%f</code>	Real em ponto flutuante (float ou double)
<code>%e</code>	Número real em notação científica com o "e" minúsculo (float ou double)
<code>%E</code>	Número real em notação científica com o "E" maiúsculo (float ou double)
<code>%%</code>	Imprimir o próprio caractere %



Adaptação do material elaborado pelo professor
João Pimentel.



Até a próxima aula...

OBRIGADO!

PROF. WANDERSON SANTOS



WANDERSON.SANTOS@PROJECABR



/ PROFESSOR WANDERSON SANTOS



/WANDERSONPS



/WANDERSONPS



/WANDERSONPS



PROFESSOR@WANDERSONSANTOS.COM
